

Assignment 02, Part 1: word_count.py

Name: Devi Aditya Dantuluri

Word counting on Jane Austen's *Pride and Prejudice* with PySpark. Questions 01 to 04 are answered in this notebook; questions 05 and 06 are in the `word_count_batch_submit` notebook.

Question 01: update the program with the data loader from Assignment 01

The original script reads a local path (`./data/gutenberg_books/1342-0.txt`). Below, the same download loader from Assignment 01 is used: `urllib.request.urlretrieve` fetches the novel from Project Gutenberg and saves it under `gutenberg_books/1342-0.txt` , which is the path the rest of the program reads.

```
In [1]: import os, glob, shutil, time
import urllib.request

os.makedirs("gutenberg_books", exist_ok=True)
file_name = "gutenberg_books/1342-0.txt"

# Data loader from Assignment 01: download Pride and Prejudice from Project Gutenberg.
url = "https://www.gutenberg.org/files/1342/1342-0.txt"
try:
    urllib.request.urlretrieve(url, file_name)
    print("Downloaded from Project Gutenberg ->", file_name)
except Exception as e:
    # Fallback: same file from the course's GitHub data repository.
    mirror = "https://raw.githubusercontent.com/jonesberg/DataAnalysisWithPythonAndPySpark-Data/trunk/gutenebe
    urllib.request.urlretrieve(mirror, file_name)
    print("gutenberg.org unreachable; downloaded the same file from the GitHub mirror ->", file_name)

print("File size (bytes):", os.path.getsize(file_name))
```

gutenberg.org unreachable; downloaded the same file from the GitHub mirror -> gutenberg_books/1342-0.txt
File size (bytes): 711299

```
In [2]: from pyspark.sql import SparkSession
from pyspark.sql.functions import col, explode, lower, regexp_extract, split

# Create the SparkSession (entry point). appName is just a label shown in the Spark UI.
spark = (SparkSession.builder
        .appName("Analyzing the vocabulary of Pride and Prejudice.")
        # Adaptive Query Execution is ON by default in Spark 3.x/4.x and automatically merges small shuffle
        # partitions into one for a tiny dataset like this. It is switched OFF here so that the effect of
        # coalesce() on the number of partitions/files is visible (see the note at the end of Question 02).
        .config("spark.sql.adaptive.enabled", "false")
        .getOrCreate())
spark.sparkContext.setLogLevel("ERROR")

print("Spark version:", spark.version)
print("Default shuffle partitions:", spark.conf.get("spark.sql.shuffle.partitions"))
print("Adaptive query execution:", spark.conf.get("spark.sql.adaptive.enabled"))
```

Spark version: 4.2.0
Default shuffle partitions: 200
Adaptive query execution: false

Study: how `groupby()` and `count()` work on data objects

Python is object-oriented: every step returns a **new DataFrame object** and the next method is called on that object. `groupby("word")` does not return a DataFrame; it returns a `GroupedData` object that only knows how the rows are partitioned into groups. Calling an aggregation such as `count()` on that object collapses each group into one row and returns a DataFrame with the grouping column plus a `count` column. Nothing is computed until an action (`show` , `count` , `write`) is called; the chain only builds a plan.

```
In [3]: # Small example to see the object types and the effect of groupby().count().
toy = spark.createDataFrame([("the",), ("cat",), ("the",), ("dog",), ("the",), ("cat",)], ["word"])
print("toy is a      :", type(toy))
grouped = toy.groupby("word")
print("toy.groupby() is a:", type(grouped))
counted = grouped.count()
print("grouped.count() is:", type(counted))
counted.orderBy("count", ascending=False).show()

# Contrast: DataFrame.count() with no groupby is an ACTION that returns a single Python integer (number of rows)
print("toy.count() returns", toy.count(), "of type", type(toy.count()))

toy is a      : <class 'pyspark.sql.classic.dataframe.DataFrame'>
toy.groupby() is a: <class 'pyspark.sql.group.GroupedData'>
grouped.count() is: <class 'pyspark.sql.classic.dataframe.DataFrame'>
+----+-----+
|word|count|
+----+-----+
| the|    3|
| cat|    2|
| dog|    1|
+----+-----+

toy.count() returns 6 of type <class 'int'>
```

Question 02: display the result after each operation in the chain

The `book` object is transformed step by step. Each `select / where` returns a new DataFrame; the intermediate objects are shown with `show()`.

```
In [4]: # 1) Read the text file: one row per line, single column "value".
book = spark.read.text(file_name)
print("Rows (lines):", book.count(), "| input partitions:", book.rdd.getNumPartitions())
book.show(10, truncate=False)

Rows (lines): 13427 | input partitions: 1
+-----+-----+
|value|
+-----+-----+
|The Project Gutenberg EBook of Pride and Prejudice, by Jane Austen|
|
|This eBook is for the use of anyone anywhere at no cost and with|
|almost no restrictions whatsoever. You may copy it, give it away or|
|re-use it under the terms of the Project Gutenberg License included|
|with this eBook or online at www.gutenberg.org|
|
|
|Title: Pride and Prejudice|
|
+-----+-----+
only showing top 10 rows
```

```
In [5]: # 2) split each line on spaces -> array of tokens per row.
lines = book.select(split(book.value, " ").alias("line"))
lines.show(10, truncate=False)

+-----+-----+
|line|
+-----+-----+
|[The, Project, Gutenberg, EBook, of, Pride, and, Prejudice,, by, Jane, Austen]|
|[[]]|
|[This, eBook, is, for, the, use, of, anyone, anywhere, at, no, cost, and, with]|
|[almost, no, restrictions, whatsoever., , You, may, copy, it,, give, it, away, or]|
|[re-use, it, under, the, terms, of, the, Project, Gutenberg, License, included]|
|[with, this, eBook, or, online, at, www.gutenberg.org]|
|[[]]|
|[[]]|
|[Title:, Pride, and, Prejudice]|
|[[]]|
+-----+-----+
only showing top 10 rows
```

```
In [6]: # 3) explode the arrays -> one token per row.
words = lines.select(explode(col("line")).alias("word"))
print("Rows (tokens):", words.count())
words.show(10)
```

Rows (tokens): 127368

```
+-----+
|      word|
+-----+
|      The|
| Project|
| Gutenberg|
| EBook|
|      of|
|      Pride|
|      and|
|Prejudice,|
|      by|
|      Jane|
+-----+
```

only showing top 10 rows

```
In [7]: # 4) lowercase every token.
words_lower = words.select(lower(col("word")).alias("word"))
words_lower.show(10)
```

```
+-----+
|      word|
+-----+
|      the|
| project|
| gutenberg|
| ebook|
|      of|
|      pride|
|      and|
|prejudice,|
|      by|
|      jane|
+-----+
```

only showing top 10 rows

```
In [8]: # 5) regexp_extract with "[a-z']*": keep the leading run of lowercase letters and apostrophes
#      (the apostrophe is now allowed, so "don't" and "elizabeth's" survive intact, unlike Assignment 01).
words_clean = words_lower.select(regexp_extract(col("word"), "[a-z']* ", 0).alias("word"))
words_clean.show(10)
```

```
+-----+
|      word|
+-----+
|      the|
| project|
|gutenberg|
|      ebook|
|      of|
|      pride|
|      and|
|prejudice|
|      by|
|      jane|
+-----+
```

only showing top 10 rows

```
In [9]: # 6) drop the empty strings produced by tokens that start with a non-letter.
words_nonull = words_clean.where(col("word") != "")
print("Rows before where():", words_clean.count(), "| after:", words_nonull.count())
words_nonull.show(10)
```

Rows before where(): 127368 | after: 122175

```
+-----+
|      word|
+-----+
|      the|
| project|
|gutenberg|
|  ebook|
|    of|
|  pride|
|    and|
|prejudice|
|    by|
|    jane|
+-----+
```

only showing top 10 rows

```
In [10]: # 7) groupby + count -> one row per distinct word with its frequency.
results = words_nonnull.groupby(col("word")).count()
print("Distinct words:", results.count())
print("Partitions of results:", results.rdd.getNumPartitions())
results.show(10)
```

Distinct words: 6595

Partitions of results: 200

```
+-----+-----+
|      word|count|
+-----+-----+
|  online|    4|
|   some|  203|
|   still|   72|
|    few|   72|
|   hope|  122|
|  those|   60|
| cautious|   4|
| lady's|    8|
|imitation|   1|
|    art|    3|
+-----+-----+
```

only showing top 10 rows

```
In [11]: # 8) Sort by frequency, descending, and show the 10 most common words.
results.orderBy("count", ascending=False).show(10)
```

```
+----+-----+
|word|count|
+----+-----+
| the| 4480|
|  to| 4218|
|  of| 3711|
| and| 3504|
| her| 2199|
|  a| 1982|
| in| 1909|
| was| 1838|
|  i| 1749|
| she| 1668|
+----+-----+
```

only showing top 10 rows

What does `coalesce(1)` do in this context?

Spark keeps a DataFrame split into **partitions**, and when it writes to disk it writes **one file per partition** (a `part-00000-...csv`, `part-00001-...csv`, and so on inside the output *folder*). After `groupby().count()`, the data has been shuffled, so `results` has as many partitions as the shuffle produced. `coalesce(1)` **merges all of those partitions into a single partition without a full shuffle**, so the writer produces exactly **one CSV file**. It is used here purely for convenience: one human-readable output file instead of many fragments. The cost is that a single task has to write everything, so it cannot be parallelized (fine for a small novel, a bottleneck for big data). `coalesce(n)` can only **reduce** the number of partitions; it never increases it (`repartition(n)` does that, with a full shuffle).

```
In [12]: # Helper: write with a given coalesce setting and report how many part files were created.
```

```
def write_and_count_files(df, out_dir, n_coalesce=None):
    shutil.rmtree(out_dir, ignore_errors=True)
    if n_coalesce is None:
        df.write.csv(out_dir)                # no coalesce
    else:
        df.coalesce(n_coalesce).write.csv(out_dir)
        part_files = sorted(glob.glob(os.path.join(out_dir, "part-*")))
        print(f"{out_dir}: {len(part_files)} data file(s) written")
        for p in part_files[:6]:
            print(" ", os.path.basename(p), f"({os.path.getsize(p)} bytes)")
        if len(part_files) > 6:
            print(" ...")
        return len(part_files)

# a) With coalesce(1), as in the original script -> one file.
print("results has", results.rdd.getNumPartitions(), "partitions before coalesce")
write_and_count_files(results, "simple_count_single_partition.csv", n_coalesce=1)
```

results has 200 partitions before coalesce
 simple_count_single_partition.csv: 1 data file(s) written
 part-00000-d0bba5bf-41ea-4393-add4-f28b1fcb94a8-c000.csv (70993 bytes)

Out[12]: 1

```
In [13]: # b) Without coalesce: one file per partition of `results`.
n_parts = results.rdd.getNumPartitions()
print("Number of data partitions in results:", n_parts)
write_and_count_files(results, "simple_count_no_coalesce.csv", n_coalesce=None)
```

Number of data partitions in results: 200
 simple_count_no_coalesce.csv: 200 data file(s) written
 part-00000-b66b92cf-30bb-4002-ac1b-a237009f52e6-c000.csv (468 bytes)
 part-00001-b66b92cf-30bb-4002-ac1b-a237009f52e6-c000.csv (389 bytes)
 part-00002-b66b92cf-30bb-4002-ac1b-a237009f52e6-c000.csv (204 bytes)
 part-00003-b66b92cf-30bb-4002-ac1b-a237009f52e6-c000.csv (367 bytes)
 part-00004-b66b92cf-30bb-4002-ac1b-a237009f52e6-c000.csv (241 bytes)
 part-00005-b66b92cf-30bb-4002-ac1b-a237009f52e6-c000.csv (251 bytes)
 ...

Out[13]: 200

```
In [14]: # c) With coalesce(4): the partitions are merged down to 4 -> 4 files.
print("Partitions after coalesce(4):", results.coalesce(4).rdd.getNumPartitions())
write_and_count_files(results, "simple_count_four_partitions.csv", n_coalesce=4)
```

Partitions after coalesce(4): 4
 simple_count_four_partitions.csv: 4 data file(s) written
 part-00000-3cf2a484-5aa9-4227-802d-395ae0a162a3-c000.csv (17544 bytes)
 part-00001-3cf2a484-5aa9-4227-802d-395ae0a162a3-c000.csv (17339 bytes)
 part-00002-3cf2a484-5aa9-4227-802d-395ae0a162a3-c000.csv (18108 bytes)
 part-00003-3cf2a484-5aa9-4227-802d-395ae0a162a3-c000.csv (18002 bytes)

Out[14]: 4

```
In [15]: # Confirm that every variant contains the same data (same number of rows), just split differently.
for d in ["simple_count_single_partition.csv", "simple_count_no_coalesce.csv", "simple_count_four_partitions.csv"]:
    print(f"{d:40s} rows = {spark.read.csv(d).count()}")
```

simple_count_single_partition.csv	rows = 6595
simple_count_no_coalesce.csv	rows = 6595
simple_count_four_partitions.csv	rows = 6595

Explanation of the file counts

- **Without coalesce** : PySpark writes one file per data partition, so the number of `part-*.csv` files equals the number of partitions of `results` printed above, which is **200**. That number comes from the shuffle performed by `groupby` : `spark.sql.shuffle.partitions` defaults to 200, so the 6,595 distinct words are hashed into 200 buckets and each bucket becomes one small file (a few hundred bytes each). For a novel this is wasteful: 200 tiny files for 70 KB of results.
- **With coalesce(4)** : the 200 partitions are combined into 4 partitions (each new partition absorbs about 50 of the old ones), so exactly **4 files** are written, each roughly a quarter of the data.
- **With coalesce(1)** : everything is merged into a single partition, hence **1 file**.

All three folders hold the same 6,595 rows; only the physical layout differs.

Note on Adaptive Query Execution (AQE). With Spark's default settings (`spark.sql.adaptive.enabled = true`) the result is different: AQE looks at the actual shuffle sizes at run time and merges the 200 small shuffle partitions into a single partition, so `results` already has 1 partition and all three variants write exactly 1 file (`coalesce(4)` cannot increase partitions, so it is a no-op). That is demonstrated in the next cell. This is why AQE was disabled at the top of the notebook: to make the partition behaviour the assignment asks about observable.

```
In [16]: # Demonstration: the same groupby with Adaptive Query Execution ON (Spark's default).
spark.conf.set("spark.sql.adaptive.enabled", "true")
results_aqe = words_nonull.groupby(col("word")).count()
print("Partitions of results with AQE on :", results_aqe.rdd.getNumPartitions())
write_and_count_files(results_aqe, "simple_count_aqe_no_coalesce.csv", n_coalesce=None)
print("Partitions after coalesce(4) with AQE on:", results_aqe.coalesce(4).rdd.getNumPartitions(), "(coalesce
spark.conf.set("spark.sql.adaptive.enabled", "false") # back to the setting used in the rest of the notebook
```

```
Partitions of results with AQE on : 1
simple_count_aqe_no_coalesce.csv: 1 data file(s) written
  part-000000-a07e1bc8-a6cc-4672-9e18-a55d30df158e-c000.csv (70993 bytes)
Partitions after coalesce(4) with AQE on: 1 (coalesce cannot increase partitions)
```

Question 03: stop word analysis

The top of the descending list is dominated by stop words (the, to, of, and, her, a, in, was, i, she, ...). Below, the list is shown further down to see where informative (content) words start to appear.

```
In [17]: # Show the ranked list with a rank number so it is easy to see how far down informative words appear.
from pyspark.sql.functions import monotonically_increasing_id, row_number
from pyspark.sql.window import Window

ranked = (results.orderBy(col("count").desc(), col("word"))
          .withColumn("rank", row_number().over(Window.orderBy(col("count").desc(), col("word")))))
ranked.select("rank", "word", "count").show(60)
```

rank	word	count
1	the	4480
2	to	4218
3	of	3711
4	and	3504
5	her	2199
6	a	1982
7	in	1909
8	was	1838
9	i	1749
10	she	1668
11	that	1487
12	it	1482
13	not	1427
14	you	1300
15	he	1296
16	be	1257
17	his	1247
18	as	1174
19	had	1170
20	with	1092
21	for	1051
22	but	896
23	is	859
24	have	838
25	at	797
26	mr	766
27	him	753
28	on	722
29	my	650
30	by	648
31	all	633
32	elizabeth	594
33	they	587
34	so	582
35	were	562
36	which	537
37	could	524
38	been	514
39	from	499
40	very	473
41	no	469
42	would	465
43	this	463
44	their	440
45	what	434
46	me	431
47	them	429
48	your	426
49	will	407
50	said	401
51	such	389
52	darcy	374
53	or	368
54	when	355
55	an	354
56	are	350
57	do	342
58	mrs	338
59	much	327
60	there	327

only showing top 60 rows

Observation. Roughly the first 25 to 30 positions are all function words. The first clearly informative words are proper nouns and content words such as `mr`, `elizabeth`, `darcy`, `mrs`, `bennet`, `jane`, `bingley`, `miss`, which appear in the 20s to 40s. To see a list that is mostly informative, one has to go past about rank 30, and even then function words keep appearing between the content words.

In [18]: # A list of stop words: articles, pronouns, auxiliary/modal verbs, prepositions, conjunctions and other funct.

```

stop_words = [
    # articles / determiners
    "the", "a", "an", "this", "that", "these", "those", "some", "any", "every", "each", "no",
    # pronouns
    "i", "me", "my", "mine", "myself", "you", "your", "yours", "yourself", "he", "him", "his", "himself",
    "she", "her", "hers", "herself", "it", "its", "itself", "we", "us", "our", "ours", "they", "them",
    "their", "theirs", "themselves", "who", "whom", "whose", "which", "what", "one", "all", "both",
    # auxiliary / modal verbs and common "to be"/"to have"/"to do" forms
    "am", "is", "are", "was", "were", "be", "been", "being", "have", "has", "had", "having",
    "do", "does", "did", "will", "would", "shall", "should", "can", "could", "may", "might", "must",
    # prepositions
    "of", "to", "in", "on", "at", "by", "for", "with", "from", "into", "upon", "about", "as",
    "than", "over", "after", "before", "between", "through", "without", "under",
    # conjunctions and other function words
    "and", "but", "or", "nor", "so", "if", "then", "there", "here", "when", "where", "how", "why",
    "not", "very", "such", "much", "more", "most", "own", "only", "too", "also", "again",
    "out", "up", "down", "now", "ever", "never", "yet", "still", "though", "however", "well",
    "s", "t", "'s", "'",
]
print("Number of stop words:", len(stop_words))

# Exclude stop words with where() + isin(): keep rows whose word is NOT in the list.
results_no_stop = results.where(~col("word").isin(stop_words))

print("Distinct words before:", results.count(), "| after removing stop words:", results_no_stop.count())
results_no_stop.orderBy(col("count").desc()).show(20)

```

Number of stop words: 133
Distinct words before: 6595 | after removing stop words: 6464

```

+-----+-----+
|      word|count|
+-----+-----+
|      mr| 766|
|elizabeth| 594|
|      said| 401|
|      darcy| 374|
|      mrs| 338|
|      bennet| 294|
|      miss| 272|
|      jane| 266|
| bingley| 255|
|      know| 237|
|      other| 224|
|      soon| 216|
|      think| 211|
|      time| 200|
|      good| 190|
|  little| 188|
|  sister| 180|
|      lady| 176|
| nothing| 173|
|      make| 168|
+-----+-----+

```

only showing top 20 rows

Which method and at which step? The natural tool is `where()` (equivalently `filter()`) combined with `isin(stop_words)` negated with `~`. There are two sensible places in the chain:

1. **Right after `words_nonull` and before `groupby()`**, i.e.
`words_nonull.where(~col("word").isin(stop_words)).groupBy("word").count()`. This removes the stop-word rows *before* the shuffle, so the expensive groupby processes fewer rows. This is the better choice for big data.
2. **After `groupby().count()` on `results`**, as done above. It filters far fewer rows (one per distinct word instead of one per occurrence) so the filter itself is cheap, but the shuffle has already counted the stop words.

Both give identical output; option 1 is preferable in a production pipeline because it reduces the data volume earliest. Option 1 is shown below for confirmation.

```

In [19]: # Option 1: filter stop words BEFORE the groupby, in the transformation chain.
results_no_stop_early = (words_nonull
    .where(~col("word").isin(stop_words)) # stop-word removal step

```

```

        .groupBy("word").count()
results_no_stop_early.orderBy(col("count").desc()).show(20)

```

```

+-----+-----+
|   word|count|
+-----+-----+
|    mr|  766|
|elizabeth|  594|
|    said|  401|
|   darcy|  374|
|    mrs|  338|
|  bennet|  294|
|   miss|  272|
|   jane|  266|
| bingley|  255|
|   know|  237|
|  other|  224|
|   soon|  216|
|  think|  211|
|   time|  200|
|   good|  190|
| little|  188|
| sister|  180|
|   lady|  176|
| nothing|  173|
|   make|  168|
+-----+-----+

```

only showing top 20 rows

Question 04: computational cost analysis

The whole program (read, split, explode, lower, clean, filter, groupby, count, sort, write) is wrapped in a function and timed with `time.perf_counter()` for the three write variants. Each variant is run three times and the mean is reported, because JVM warm-up makes the very first run slower.

```

In [20]: def run_pipeline(path, out_dir, n_coalesce=None):
    """Run the entire word-count program end to end and return elapsed seconds."""
    shutil.rmtree(out_dir, ignore_errors=True)
    t0 = time.perf_counter()
    res = (spark.read.text(path)
        .select(split(col("value"), " ").alias("line"))
        .select(explode(col("line")).alias("word"))
        .select(lower(col("word")).alias("word"))
        .select(regexp_extract(col("word"), "[a-z']* ", 0).alias("word"))
        .where(col("word") != "")
        .groupBy(col("word")).count())
    top10 = res.orderBy("count", ascending=False).take(10)  # same action as show(10), without printing 9 ta
    if n_coalesce is None:
        res.write.csv(out_dir)
    else:
        res.coalesce(n_coalesce).write.csv(out_dir)
    return time.perf_counter() - t0

def benchmark(path, label_prefix, repeats=3):
    import pandas as pd
    variants = [("without coalesce", None), ("coalesce(1)", 1), ("coalesce(4)", 4)]
    rows = []
    for name, n in variants:
        times = []
        for r in range(repeats):
            t = run_pipeline(path, f"timing_{label_prefix}_{n}_{r}.csv", n)
            times.append(t)
        rows.append({"variant": name, "run_1_s": times[0], "run_2_s": times[1], "run_3_s": times[2],
            "mean_s": sum(times) / len(times), "mean_min": sum(times) / len(times) / 60,
            "files_written": len(glob.glob(f"timing_{label_prefix}_{n}_0.csv/part-*"))})
    return pd.DataFrame(rows).round(3)

timing_single = benchmark(file_name, "novel")
timing_single

```

Out[20]:

	variant	run_1_s	run_2_s	run_3_s	mean_s	mean_min	files_written
0	without coalesce	8.301	6.863	6.419	7.194	0.120	200
1	coalesce(1)	1.586	1.913	1.470	1.656	0.028	1
2	coalesce(4)	1.465	1.454	1.506	1.475	0.025	4

Interpretation. For a single 700 KB novel the computation itself (read, split, count) is only a fraction of a second; the differences come almost entirely from the write stage.

- **Without coalesce** is the slowest (about 7 s): Spark launches 200 write tasks, one per shuffle partition, and each task opens, writes and commits a tiny CSV file. Task scheduling and file-system overhead dominate.
- **coalesce(1)** and **coalesce(4)** are both about 1.5 to 1.7 s: 1 or 4 write tasks instead of 200, so the overhead almost disappears. With such a small result the extra parallelism of 4 files does not measurably beat 1 file.

On a genuinely large dataset the picture changes: fewer partitions means less parallelism, so `coalesce(1)` would become a bottleneck (one executor writes everything) while a moderate number of partitions such as `coalesce(4)` would balance parallelism against per-file overhead. The right partition count depends on data size and cluster size, which is what Adaptive Query Execution tries to choose automatically.

In [21]:

```
# Clean up the timing output folders.
for d in glob.glob("timing_novel_*.csv"):
    shutil.rmtree(d, ignore_errors=True)
spark.stop()
```

Assignment 02, Part 2: word_count_batch_submit.py

Name: Devi Aditya Dantuluri

Batch word counting over all the text files in the `gutenberg_books` folder with one PySpark routine. Questions 05 and 06.

Setup: download every book in the `gutenberg_books` repository

The course data repository (`jonesberg/DataAnalysisWithPythonAndPySpark-Data` , branch `trunk`) is used. The books are downloaded with the same `urllib` loader as before, into a local `gutenberg_books/` folder.

```
In [1]: import os, glob, shutil, time
import urllib.request

repo = "https://raw.githubusercontent.com/jonesberg/DataAnalysisWithPythonAndPySpark-Data/trunk/gutenberg_books"
book_files = ["11-0.txt", "1342-0.txt", "1661-0.txt", "2701-0.txt", "30254-0.txt", "84-0.txt"]

os.makedirs("gutenberg_books", exist_ok=True)
for name in book_files:
    urllib.request.urlretrieve(repo + name, os.path.join("gutenberg_books", name))

# 7th file: War and Peace by Leo Tolstoy, Project Gutenberg eBook #2600 (https://www.gutenberg.org/ebooks/2600)
wp_path = os.path.join("gutenberg_books", "2600-0.txt")
try:
    urllib.request.urlretrieve("https://www.gutenberg.org/files/2600/2600-0.txt", wp_path)
    print("War and Peace downloaded from gutenberg.org")
except Exception as e:
    # Fallback: the same Gutenberg text mirrored on GitHub (GITenberg project).
    urllib.request.urlretrieve("https://raw.githubusercontent.com/GITenberg/War-and-Peace_2600/master/2600-0.txt", wp_path)
    print("gutenberg.org unreachable; War and Peace downloaded from the GITenberg mirror")

# Print the title found in each file's Gutenberg header, and the file size.
for path in sorted(glob.glob("gutenberg_books/*.txt")):
    title = next((l.strip() for l in open(path, encoding="utf-8") if l.startswith("Title:")), "?")
    print(f"{os.path.basename(path):12s} {os.path.getsize(path):>9,d} bytes {title}")
print("\nNumber of text files:", len(glob.glob("gutenberg_books/*.txt")))
```

```
gutenberg.org unreachable; War and Peace downloaded from the GITenberg mirror
11-0.txt      169,859 bytes   Title: Alice's Adventures in Wonderland
1342-0.txt    711,299 bytes   Title: Pride and Prejudice
1661-0.txt    595,478 bytes   Title: The Adventures of Sherlock Holmes
2600-0.txt    3,359,542 bytes  Title: War and Peace
2701-0.txt    1,253,868 bytes  Title: Moby Dick; or The Whale
30254-0.txt   1,057,984 bytes  Title: The Romance of Lust A classic Victorian erotic novel
84-0.txt      442,949 bytes   Title: Frankenstein
```

Number of text files: 7

Note: the `gutenberg_books` folder in the course repository currently contains six books (`11-0` , `1342-0` , `1661-0` , `2701-0` , `30254-0` , `84-0`). To reach the seven files mentioned in the assignment, *War and Peace* by Leo Tolstoy (Project Gutenberg #2600) was added as `2600-0.txt` . It is by far the largest file (about 3.2 MB, roughly 4.5 times the size of *Pride and Prejudice*), so it dominates the batch counts. The code works unchanged for any number of files because the wildcard `*.txt` is used.

```
In [2]: from pyspark.sql import SparkSession
import pyspark.sql.functions as F

spark = (SparkSession.builder
        .appName("Counting word occurrences from a book.")
        .config("spark.sql.adaptive.enabled", "false") # same setting as Part 1, so partition counts are correct
        .getOrCreate())
spark.sparkContext.setLogLevel("ERROR")
```

The batch program

All operations are chained into one expression. The only change from the original script is the input path:

`gutenberg_books/*.txt` instead of `gutenberg_books/1342-0.txt`. Spark expands the wildcard and reads every matching file as if it were one big text file (one row per line from every file).

```
In [3]: def word_counts(path):
        """The word_count_batch_submit.py chain, parameterised by the input path."""
        return (
            spark.read.text(path)
            .select(F.split(F.col("value"), " ").alias("line"))
            .select(F.explode(F.col("line")).alias("word"))
            .select(F.lower(F.col("word")).alias("word"))
            .select(F.regexp_extract(F.col("word"), "[a-z]*", 0).alias("word"))
            .where(F.col("word") != "")
            .groupby(F.col("word"))
            .count()
        )

        # Single novel (for comparison) and the whole batch.
        results_novel = word_counts("gutenberg_books/1342-0.txt")
        results_batch = word_counts("gutenberg_books/*.txt")

        print("Input partitions, single novel:", spark.read.text("gutenberg_books/1342-0.txt").rdd.getNumPartitions())
        print("Input partitions, batch      :", spark.read.text("gutenberg_books/*.txt").rdd.getNumPartitions())

Input partitions, single novel: 1
Input partitions, batch      : 1
```

```
In [4]: # Write the batch results to a single-partition file, exactly as in the script.
        shutil.rmtree("results_single_partition.csv", ignore_errors=True)
        results_batch.coalesce(1).write.csv("results_single_partition.csv")
        print(os.listdir("results_single_partition.csv"))

['.SUCCESS.crc', '.SUCCESS', 'part-00000-b2e8ec2d-5634-411f-8023-524082046a53-c000.csv', '.part-00000-b2e8ec2d-5634-411f-8023-524082046a53-c000.csv.crc']
```

Question 05: does the single-partition file contain results from all files?

```
In [5]: # Read the written file back and compare it with the counts computed directly from the DataFrames.
        written = spark.read.csv("results_single_partition.csv").toDF("word", "count").withColumn("count", F.col("count"))

        lines_novel = spark.read.text("gutenberg_books/1342-0.txt").count()
        lines_batch = spark.read.text("gutenberg_books/*.txt").count()
        lines_sum = sum(spark.read.text(p).count() for p in sorted(glob.glob("gutenberg_books/*.txt")))

        print("Lines read, single novel :", lines_novel)
        print("Lines read, batch (*.txt):", lines_batch, "| sum over the individual files:", lines_sum)
        print()
        print("Distinct words, novel      :", results_novel.count())
        print("Distinct words, batch      :", results_batch.count())
        print("Rows in written CSV file:", written.count())
        print()
        print("Total word occurrences, novel:", results_novel.agg(F.sum("count")).first()[0])
        print("Total word occurrences, batch:", results_batch.agg(F.sum("count")).first()[0])
        print("Total word occurrences, file :", written.agg(F.sum("count")).first()[0])

Lines read, single novel : 13427
Lines read, batch (*.txt): 143965 | sum over the individual files: 143965

Distinct words, novel      : 6595
Distinct words, batch      : 29042
Rows in written CSV file: 29042

Total word occurrences, novel: 122175
Total word occurrences, batch: 1291358
Total word occurrences, file : 1291358
```

Confirmation. Yes. The number of lines read with `*.txt` equals the sum of the lines of the individual files, the batch result has far more distinct words than the novel alone, and the single-partition CSV has exactly the same number of rows and the same total count as the batch DataFrame. `coalesce(1)` only changes how the output is laid out on disk (one file instead of many); it does not drop any data, so the file holds the counts and, after sorting, the ranking

for all the books together.

```
In [6]: # Direct evidence: words that appear in the batch file but not in Pride and Prejudice at all.
# These can only come from the other books (Moby Dick, Frankenstein, Alice, Sherlock Holmes, War and Peace, .
probe = ["whale", "ahab", "holmes", "watson", "alice", "hatter", "frankenstein", "monster", "darcy", "elizabeth",
        "pierre", "natasha", "napoleon", "moscow", "kutúzov", "prince"]
cmp = (results_novel.withColumnRenamed("count", "novel_count")
      .join(results_batch.withColumnRenamed("count", "batch_count"), "word", "full")
      .join(written.withColumnRenamed("count", "file_count"), "word", "full")
      .where(F.col("word").isin(probe))
      .fillna(0)
      .orderBy(F.col("batch_count").desc()))
cmp.show(truncate=False)
```

word	novel_count	batch_count	file_count
pierre	0	1946	1946
prince	0	1906	1906
whale	0	1235	1235
moscow	0	716	716
elizabeth	594	685	685
napoleon	0	576	576
ahab	0	504	504
holmes	0	460	460
alice	0	409	409
darcy	374	374	374
monster	0	85	85
watson	0	80	80
hatter	0	56	56
frankenstein	0	31	31

Differences between batch processing and the single novel

Top-10 lists side by side, and the words whose rank changes most.

```
In [7]: print("Top 10, single novel (Pride and Prejudice):")
results_novel.orderBy("count", ascending=False).show(10)
print("Top 10, batch of all books (from the single-partition file):")
written.orderBy("count", ascending=False).show(10)
```

Top 10, single novel (Pride and Prejudice):

word	count
the	4480
to	4218
of	3711
and	3504
her	2199
a	1982
in	1909
was	1838
i	1749
she	1668

only showing top 10 rows

Top 10, batch of all books (from the single-partition file):

```
+----+----+
|word|count|
+----+----+
| the|73153|
| and|45737|
| to|37156|
| of|36162|
| a|24864|
| in|21658|
| i|17577|
|that|17525|
| was|16243|
```

```
| he|16152|
+----+----+
```

only showing top 10 rows

```
In [8]: # Same word, different counts: the top words are the same stop words, but the counts are several times larger
# in the batch, and the order changes (e.g. "the"/"of"/"and" dominate the batch while "to"/"her" fall back:
# Pride and Prejudice is dialogue-heavy with a female protagonist, Moby Dick is descriptive prose).
top_cmp = (results_novel.withColumnRenamed("count", "novel_count")
            .join(results_batch.withColumnRenamed("count", "batch_count"), "word", "outer")
            .fillna(0)
            .withColumn("ratio_batch_over_novel", F.round(F.col("batch_count") / F.greatest(F.col("novel_count"), 1), 1)))
top_cmp.orderBy(F.col("batch_count").desc()).show(20)
```

```
+----+----+----+----+
|word|novel_count|batch_count|ratio_batch_over_novel|
+----+----+----+----+
| the|      4480|      73153|             16.3|
| and|      3504|      45737|             13.1|
| to|      4218|      37156|              8.8|
| of|      3711|      36162|              9.7|
| a|       1982|      24864|             12.5|
| in|       1909|      21658|             11.3|
| i|       1749|      17577|             10.0|
|that|       1487|      17525|             11.8|
| was|       1838|      16243|              8.8|
| he|       1296|      16152|             12.5|
| his|       1247|      14596|             11.7|
| it|       1482|      14422|              9.7|
| her|       2199|      12636|              5.7|
|with|       1092|      12251|             11.2|
| had|       1170|      11069|              9.5|
| as|       1174|      10354|              8.8|
| you|       1300|       9795|              7.5|
| not|       1427|       9451|              6.6|
| at|        797|       9193|             11.5|
| she|       1668|       9005|              5.4|
+----+----+----+----+
```

only showing top 20 rows

```
In [9]: # Informative words after removing the same stop-word list as Part 1: the batch top-20 is a mixture of
# characters/themes from all books instead of Austen's characters only.
stop_words = [
    "the", "a", "an", "this", "that", "these", "those", "some", "any", "every", "each", "no",
    "i", "me", "my", "mine", "myself", "you", "your", "yours", "yourself", "he", "him", "his", "himself",
    "she", "her", "hers", "herself", "it", "its", "itself", "we", "us", "our", "ours", "they", "them",
    "their", "theirs", "themselves", "who", "whom", "whose", "which", "what", "one", "all", "both",
    "am", "is", "are", "was", "were", "be", "been", "being", "have", "has", "had", "having",
    "do", "does", "did", "will", "would", "shall", "should", "can", "could", "may", "might", "must",
    "of", "to", "in", "on", "at", "by", "for", "with", "from", "into", "upon", "about", "as",
    "than", "over", "after", "before", "between", "through", "without", "under",
    "and", "but", "or", "nor", "so", "if", "then", "there", "here", "when", "where", "how", "why",
    "not", "very", "such", "much", "more", "most", "own", "only", "too", "also", "again",
    "out", "up", "down", "now", "ever", "never", "yet", "still", "though", "however", "well",
    "s", "t", "s", "t",
]
print("Top 20 informative words, single novel:")
results_novel.where(~F.col("word").isin(stop_words)).orderBy("count", ascending=False).show(20)
print("Top 20 informative words, batch of all books:")
results_batch.where(~F.col("word").isin(stop_words)).orderBy("count", ascending=False).show(20)
```

Top 20 informative words, single novel:

```

+-----+-----+
| word|count|
+-----+-----+
| mr| 766|
| elizabeth| 594|
| said| 401|
| darcy| 374|
| mrs| 338|
| bennet| 294|
| miss| 272|
| jane| 266|
| bingley| 255|
| know| 237|
| other| 224|
| soon| 216|
| think| 211|
| time| 200|
| good| 190|
| little| 188|
| sister| 180|
| lady| 176|
| nothing| 173|
| make| 168|
+-----+-----+

```

only showing top 20 rows

Top 20 informative words, batch of all books:

```

+-----+-----+
| word|count|
+-----+-----+
| said| 4933|
| man| 2377|
| time| 2239|
| pierre| 1946|
| prince| 1906|
| other| 1844|
| see| 1772|
| like| 1763|
| little| 1758|
| know| 1691|
| first| 1652|
| two| 1555|
| old| 1492|
| thought| 1396|
| long| 1377|
| eyes| 1372|
| way| 1335|
| went| 1318|
| room| 1309|
| go| 1303|
+-----+-----+

```

only showing top 20 rows

What changes with batch processing.

- The vocabulary grows from 6,595 distinct words to 29,042 (about 4.4x), and total occurrences grow from 122,175 to 1,291,358 (about 10.6x).
- Words absent from *Pride and Prejudice* (pierre , prince , whale , moscow , napoleon , ahab , holmes , alice , ...) appear with high counts; they come from the other files, which confirms the batch read.
- The stop words stay at the top but their relative order shifts (and overtakes to and of ; he and his enter the top 10 while her and she fall out, because *War and Peace* and *Moby Dick*, the two largest books, are centred on male characters).
- After stop-word removal the novel's top words are Austen's characters (mr , elizabeth , darcy , bennet), while the batch top words are dominated by the largest books (pierre and prince from *War and Peace* enter the top 5, and generic narrative words such as man , see , like , old , eyes , room take over), so the single-file ranking is no longer representative of any one novel.

Question 06: computational cost analysis for batch processing

Same benchmark as Question 04 (three runs per variant, mean reported), now with `gutenberg_books/*.txt` as the input.

```
In [10]: def run_pipeline(path, out_dir, n_coalesce=None):
    shutil.rmtree(out_dir, ignore_errors=True)
    t0 = time.perf_counter()
    res = word_counts(path)
    top10 = res.orderBy("count", ascending=False).take(10)
    if n_coalesce is None:
        res.write.csv(out_dir)
    else:
        res.coalesce(n_coalesce).write.csv(out_dir)
    return time.perf_counter() - t0

def benchmark(path, label_prefix, repeats=3):
    import pandas as pd
    variants = [("without coalesce", None), ("coalesce(1)", 1), ("coalesce(4)", 4)]
    rows = []
    for name, n in variants:
        times = [run_pipeline(path, f"timing_{label_prefix}_{n}_{r}.csv", n) for r in range(repeats)]
        rows.append({"variant": name, "run_1_s": times[0], "run_2_s": times[1], "run_3_s": times[2],
                     "mean_s": sum(times) / len(times), "mean_min": sum(times) / len(times) / 60,
                     "files_written": len(glob.glob(f"timing_{label_prefix}_{n}_0.csv/part-*"))})
    return pd.DataFrame(rows).round(3)

print("Single novel (repeat of Question 04 in this session, for a like-for-like comparison):")
timing_novel = benchmark("gutenberg_books/1342-0.txt", "novel")
display(timing_novel)

print("Batch of all books:")
timing_batch = benchmark("gutenberg_books/*.txt", "batch")
display(timing_batch)
```

Single novel (repeat of Question 04 in this session, for a like-for-like comparison):

	variant	run_1_s	run_2_s	run_3_s	mean_s	mean_min	files_written
0	without coalesce	8.547	7.203	6.843	7.531	0.126	200
1	coalesce(1)	1.355	1.507	1.201	1.354	0.023	1
2	coalesce(4)	1.522	1.470	1.625	1.539	0.026	4

Batch of all books:

	variant	run_1_s	run_2_s	run_3_s	mean_s	mean_min	files_written
0	without coalesce	7.767	8.445	7.934	8.049	0.134	200
1	coalesce(1)	2.669	2.802	2.874	2.782	0.046	1
2	coalesce(4)	2.929	2.989	2.633	2.850	0.048	4

```
In [11]: # Side-by-side summary.
import pandas as pd
summary = pd.DataFrame({
    "variant": timing_novel["variant"],
    "novel_mean_s": timing_novel["mean_s"],
    "batch_mean_s": timing_batch["mean_s"],
})
summary["batch / novel"] = (summary["batch_mean_s"] / summary["novel_mean_s"]).round(2)
summary
```

```
Out[11]:
```

	variant	novel_mean_s	batch_mean_s	batch / novel
0	without coalesce	7.531	8.049	1.07
1	coalesce(1)	1.354	2.782	2.05
2	coalesce(4)	1.539	2.850	1.85

Interpretation. The batch input is about 10.6 times larger (7.6 MB vs 0.7 MB, 143,965 lines vs 13,427), yet Spark still reads it as a single input partition because all seven files together are far below the default 128 MB partition size.

The read/split/explode/count stages and the shuffle therefore do more work on one core, and the `coalesce(1)` and `coalesce(4)` runs grow from about 1.4 to 1.5 s to about 2.8 s (roughly 2x), far less than 10x, because a large share of each run is fixed Spark overhead (job scheduling, task launch, file commit) that does not depend on data size.

Without `coalesce` the batch takes about 8 s versus 7.5 s for the single novel: that variant is dominated by launching and committing 200 write tasks, which costs the same regardless of how much data is in each file. The ranking of the variants is the same in both cases: no coalesce is slowest; `coalesce(1)` and `coalesce(4)` are close, with `coalesce(1)` marginally faster on this small data because a single writer avoids extra task overhead.

```
In [12]: for d in glob.glob("timing_**.csv"):
          shutil.rmtree(d, ignore_errors=True)
          spark.stop()
```